

■ Design a Message Queue (Kafka / RabbitMQ)

System Design Interview | Asynchronous Systems

algomaster.io | Deep Dive Notes

■ Key Requirements

Functional

Producers publish messages; Consumers subscribe and process; At-least-once / exactly-once delivery; Message ordering; Replay; Dead-letter queue

Non-Functional

High throughput (millions of msg/sec); Low latency (<10ms); Durable (survive broker restarts); Horizontally scalable; Multi-consumer support

Scale

Kafka at LinkedIn: 7 trillion messages/day; partitioning is the key to horizontal scale; retention up to weeks/months

■ Core Architecture (Kafka Model)

- Topic: logical channel; split into N Partitions (ordered, immutable log); each partition replicated across brokers
- Producer: writes to partition via key hash (same key → same partition → ordering guaranteed per key)
- Consumer Group: each partition consumed by exactly one consumer in a group; add consumers to scale out (up to N = partition count)
- Broker: stores partition replicas; leader handles reads/writes; followers replicate; ZooKeeper/KRaft manages leader election
- Offset: each message has monotonic offset per partition; consumer commits offset to track progress; enables replay by resetting offset

■ Delivery Guarantees Deep Dive

- **At-Most-Once:** Producer fire-and-forget (acks=0); consumer auto-commits before processing; fast but may lose messages
- **At-Least-Once:** Producer waits for leader ack (acks=1); consumer commits after processing; may duplicate on retry — most common
- **Exactly-Once:** Idempotent producer (enable.idempotence=true) + transactional API; producer gets PID+seq; broker deduplicates; highest overhead
- **Consumer Idempotency:** Even with at-least-once, make consumers idempotent: check `processed_message_ids` in DB/Redis before acting

■ Storage & Retention

- Append-only log on disk: sequential writes = fast (disk I/O optimized); Kafka achieves 1M+ msg/sec per broker
- Page cache: OS page cache used aggressively; zero-copy `sendfile()` syscall for consumer reads → avoids user-space copy
- Retention: time-based (7 days default) or size-based (e.g., 100GB/partition); consumers can replay from any offset

- Log compaction: keep only latest message per key; enables 'current state' semantics (like a KV store); used for changelog topics
- Segment files: partition split into segment files (1GB default); old segments deleted per retention policy

■ Kafka vs RabbitMQ

Kafka	RabbitMQ
✓ Pull-based consumers	✓ Push-based consumers
✓ Log retention + replay	✓ Messages deleted on ACK
✓ High throughput (M/sec)	✓ Lower throughput (100K/sec)
✓ Ordering per partition	✓ Flexible routing (exchanges)

■ Partitioning & Ordering Strategy

- Partition by entity key (e.g., user_id, order_id) → all events for same entity in same partition → ordered processing
- Too few partitions → bottleneck; too many → overhead (each partition = file handles, memory); rule of thumb: 10-30× brokers
- Consumer lag monitoring: if lag grows → add consumers (up to partition count) or optimize consumer processing speed
- Rebalancing: adding/removing consumers triggers partition reassignment; use sticky assignor to minimize movement

■ Dead Letter Queue (DLQ) & Error Handling

- After N retries with exponential backoff, publish failed message to DLQ topic for manual inspection / replay
- Poison pill: a single bad message that always fails; DLQ prevents it from blocking entire partition
- Alert on DLQ growth; ops team investigates; fix bug → replay DLQ messages back to original topic
- Schema registry (Confluent): enforce Avro/Protobuf schema on publish; prevents malformed messages at source